

CSC490 A6

Benson Li 1007815376
Yuchen Wang 1010441595
Xuanyi Lyu 1009343266
Zheyu Zheng 1008979580

November 28, 2025

Part One - Showing Code Coverage

We have implemented an automated testing pipeline using `pytest` and `pytest-cov`. And we have configured a GitHub Actions workflow to automatically run tests and generate coverage reports upon every Pull Request. The status of our code coverage is visibly displayed via a badge on our project's README file.

- **Project Repository:** <https://github.com/UofT-CSC490-F2025/TweetVerify>

src	100% code coverage	20 minutes ago
terraform	fix	2 weeks ago
tests	100% code coverage	20 minutes ago
.gitignore	A6 Add coverage workflow	1 hour ago
README.md	FIX: Fix coverage workflow and tests	25 minutes ago
coverage.svg	Updated coverage.svg	16 minutes ago
requirements.txt	Add missing dependencies	1 hour ago

 README  

TweetVerify

coverage 100%



A deep learning-based Twitter tweet authenticity verification system supporting multiple model training and real-time prediction.

Project Overview



github-actions bot commented 5 minutes ago

Contributor ...

Coverage 100%

► Coverage Report

Tests	Skipped	Failures	Errors	Time
82	0 	0 	0 	13.675s 



Part Two: Test Coverage Breadth

Below is the execution log generated by our testing suite, demonstrating that all tests passed and confirming the coverage statistics across all modules.

```

===== tests coverage =====
----- coverage: platform linux, python 3.10.19-final-0 -----

```

Name	Stmts	Miss	Branch	BrPart	Cover	Missing
src/app_wrapper.py	11	0	2	0	100%	
src/apps/app.py	268	0	84	0	100%	
src/data_ingestion/llm_db.py	27	0	0	0	100%	
src/data_ingestion/main_db.py	29	0	0	0	100%	
src/data_ingestion/twitter_db.py	31	0	2	0	100%	
src/data_ingestion/twitter_scrape.py	34	0	12	0	100%	
src/data_preprocessing/processor.py	68	0	24	0	100%	
src/dataloader/bertdataset.py	15	0	0	0	100%	
src/dataloader/featuredataset.py	17	0	0	0	100%	
src/evaluator/evaluator.py	44	0	8	0	100%	
src/inference/predictor.py	69	0	14	0	100%	
src/model/bert.py	20	0	4	0	100%	
src/model/deberta.py	20	0	0	0	100%	
src/model/lstm.py	28	0	4	0	100%	
src/model/rnn.py	29	0	4	0	100%	
src/model/roberta.py	20	0	0	0	100%	
src/model/roberta_extra.py	29	0	2	0	100%	
src/plotter/plotter.py	21	0	0	0	100%	
src/run.py	6	0	2	0	100%	
src/security/__init__.py	3	0	0	0	100%	
src/security/input_validator.py	167	0	68	0	100%	
src/security/rate_limiter.py	111	0	24	0	100%	
src/train.py	137	0	14	0	100%	

src/train_model.py	13	0	2	0	100%
src/trainer/trainer.py	93	0	22	0	100%
src/utls/canonical_id.py	4	0	0	0	100%
src/utls/collate_batch.py	11	0	2	0	100%
src/utls/convert_indices.py	13	0	6	0	100%
src/utls/get_from_s3.py	20	0	0	0	100%
src/utls/seed.py	12	0	0	0	100%

TOTAL	1370	0	300	0	100%

Coverage XML written to file coverage.xml

===== 138 passed, 5 warnings in 15.72s =====

Part Three: Test Coverage Depth

We selected the `src/security/input_validator.py` module. This module is responsible for sanitizing user inputs, validating file uploads, enforcing schema constraints on API requests, and preventing common web vulnerabilities such as SQL Injection (SQLi), Cross-Site Scripting (XSS), and Path Traversal attacks.

Given its role as the first line of defense, ensuring its robustness against edge cases and malicious payloads is paramount. We implemented a test suite in `tests/test_input_validator_depth.py` containing 12 unit tests. The tests cover the following categories of edge cases:

Sanitization Bypass Attempts

- **Recursive HTML Tags:** Tested `sanitize_text` with nested tags like `<scr<script>ipt>` to ensure the sanitizer is not fooled by simple stripping strategies. The function correctly escapes the input using `html.escape`.
- **Invisible Characters:** Tested for Unicode zero-width spaces (e.g., `\u200b`) which are often used to bypass keyword filters.
- **Unicode Homoglyphs:** Verified behavior with characters that look identical but have different Unicode code points (e.g., Cyrillic 'a' vs. Latin 'a'), which is crucial for preventing phishing or IDN homograph attacks.

SQL Injection Failure Modes

- **Recursive Replacement Vulnerability:** We identified and documented a limitation in `sanitize_sql_input`. The function performs a single-pass replacement. We created a test case `test_sanitize_sql_input_recursive_bypass` with input `'xpxp_'` (targeting the banned `'xp_'` token). The sanitizer removes the inner instance, resulting in `'xp_'`, demonstrating a potential failure mode if used as the sole defense.
- **Unicode Quotes:** Tested inputs with Unicode apostrophes (e.g., `U+02BC`) which some databases might normalize to SQL single quotes, potentially bypassing ASCII-only filters.

Filename and Path Safety

- **Windows Reserved Names:** Tested cross-platform risks by validating filenames like `CON.txt`. While valid on Linux, these are reserved device names on Windows. The current validator allows them, which is an important edge case to document for cross-platform deployments.
- **Extreme Length:** Verified proper truncation for filenames exceeding filesystem limits (255 bytes).
- **Degenerate Inputs:** Tested filenames consisting solely of dots or spaces (e.g., "..."), ensuring they are handled gracefully (resulting in an empty string rather than an error).

Resource Exhaustion (DoS)

- **File Size Deception:** Simulated a "Zip Bomb" scenario using a Mock file object where `seek/tell` reports a massive size, ensuring the `validate_file_upload` function correctly rejects files exceeding the limit before processing.

Part Four: Two Memorable Bugs

Below is the walkthrough video and the corresponding fixes and test cases for the two most memorable bugs encountered during development.

Video Walkthrough

[Video walkthrough of both bugs](#)

Bug 1

Fix: [Commit link](#)

Test Cases: [Commit link](#)

Bug 2

Fix: [Commit link](#)

Test Cases: [Commit link](#)